

Columnas calculadas y columnas computadas persistentes

Hay un patrón que se repite constantemente en el día a día de cualquier analista. Necesitas consultar productos y, además del precio y el costo que ya están en la tabla, necesitas calcular el margen de ganancia. Entonces escribes la fórmula: precio menos costo, dividido por precio, multiplicado por cien. La primera vez no hay problema. Pero esa fórmula empieza a aparecer en un segundo script, después en un tercero, y de pronto la tienes copiada en diez o veinte consultas distintas.

¿Qué pasa el día que cambia la lógica del cálculo? Que tienes que ir consulta por consulta actualizando la fórmula. Y si se te escapa una, tu reporte va a dar números diferentes al de tu compañero, y ninguno de los dos va a saber cuál es el correcto. Es un problema que parece menor hasta que te pasa.

La solución es mover esa fórmula a la tabla misma, como una **columna calculada**. La fórmula vive en un solo lugar, y cualquier consulta que haga `SELECT * FROM Productos` automáticamente la incluye. Si mañana cambia la lógica, la actualizas una vez y todos los scripts que la usan obtienen el valor correcto sin tocar nada.

Columnas calculadas:

Una columna calculada se agrega a una tabla con `ALTER TABLE ... ADD`, pero en vez de definir un tipo de dato fijo, defines una expresión — una fórmula que referencia otras columnas de la misma tabla. Por ejemplo, puedes agregar `PrecioConIVA` como `Precio * 1.21`, o `MargenPorcentaje` como la fórmula que mencionamos al inicio.

Lo interesante es cómo funciona por debajo. La columna calculada no almacena ningún número. Lo que guarda es la fórmula. Cada vez que alguien consulta la tabla, SQL Server ejecuta ese cálculo en el momento y te devuelve el resultado. Es como si la fórmula estuviera escrita en tu `SELECT`, pero sin que tú tengas que escribirla — ya está definida en la estructura de la tabla.

Desde el punto de vista del usuario, la columna calculada se comporta exactamente igual que cualquier otra columna. Puedes incluirla en un `SELECT`, filtrar por ella con `WHERE`, ordenar por ella. Lo único que no puedes hacer es modificarla directamente con un `UPDATE`, y tiene sentido: no tiene un valor propio, su valor viene de otros campos.

En la clase vimos que al explorar la tabla en SQL Server Management Studio, las columnas calculadas aparecen con un ícono distinto y la leyenda "calculado" al lado del tipo de dato. Eso te permite identificarlas rápidamente cuando estás explorando una base de datos que no conoces.

¿Y si necesito cambiar la fórmula?

Acá hay un detalle que puede sorprender. Tu primer instinto sería hacer un `UPDATE` para cambiar el valor, o un `ALTER` para modificar la expresión. Pero las columnas calculadas no funcionan así. No puedes hacer `UPDATE Productos SET PrecioConIVA = Precio * 1.19` porque SQL Server te lo rechaza — es una columna calculada, no puedes escribir sobre ella.

Lo que tienes que hacer es eliminar la columna y volver a crearla con la nueva fórmula. Suena más drástico de lo que es: un `ALTER TABLE ... DROP COLUMN PrecioConIVA` seguido de un `ALTER TABLE ... ADD PrecioConIVA AS (Precio * 1.19)`. Son dos pasos, pero es la forma correcta. Y justamente porque cambiar la fórmula requiere modificar la estructura de la tabla, esta herramienta funciona mejor para cálculos cuya lógica es estable. Si la fórmula cambia cada semana, probablemente sea mejor dejarla en la consulta.

Columnas persistidas: cuando quieres guardar el resultado

Hasta acá, la columna calculada se recalcula cada vez que consultas la tabla. Para doce filas no importa, pero si tu tabla tiene cientos de miles de registros y la fórmula es compleja, ese recálculo constante puede afectar el rendimiento.

Para eso existe la opción `PERSISTED`. Cuando agregas una columna calculada con `PERSISTED`, SQL Server no solo guarda la fórmula — también **almacena el resultado en disco**. El cálculo se hace una vez, y después solo se recalcula cuando las columnas de las que depende se modifican. Si el precio o el costo de un producto cambian, SQL Server recalcula el margen de ese producto. Si no cambian, el valor ya está ahí listo para ser leído sin calcular nada.

La diferencia es sutil pero importante. Sin `PERSISTED`, el cálculo ocurre **siempre que consultas**. Con `PERSISTED`, el cálculo ocurre **solo cuando cambian los datos base**. En una tabla que se consulta mucho más de lo que se modifica — que es el caso típico de una tabla de productos — la ganancia en rendimiento puede ser significativa.

En la clase vimos que para la columna de margen porcentaje persistida, el instructor usó un CASE para manejar los casos donde el costo es nulo o el precio es cero, evitando así divisiones por cero o resultados sin sentido. Ese tipo de validación es especialmente importante en columnas persistidas, porque el valor va a quedar almacenado y cualquier error en la fórmula se propaga a todo el que consulte la tabla.

Hay un requisito técnico para PERSISTED: la expresión debe ser **determinista**. Eso significa que con los mismos valores de entrada siempre debe dar el mismo resultado. Funciones como `GETDATE()` o `NEWID()`, que devuelven algo diferente cada vez que se ejecutan, no se pueden usar en columnas persistidas.

La ventaja oculta: índices sobre columnas persistidas

Hay algo que una columna calculada normal no puede hacer pero una persistida sí: **puedes crear un índice sobre ella**. Y eso abre una puerta importante para el rendimiento.

Imagina que frecuentemente necesitas encontrar todos los productos con un margen mayor al 30%. Sin índice, SQL Server tiene que recorrer toda la tabla y calcular el margen de cada producto para comparar. Con una columna persistida e indexada, SQL Server va directo a los registros que cumplen la condición, igual que haría con cualquier otra columna indexada.

Esa combinación — columna persistida más índice — convierte lo que era un cálculo repetitivo en una consulta optimizada. Es el punto donde las columnas calculadas dejan de ser solo una conveniencia y se convierten en una herramienta real de optimización.

Cuándo usar columnas calculadas y cuándo no

La lógica para decidir es parecida a la de las vistas indexadas: no es algo que apliques a todo, sino que reservas para los casos donde el beneficio es claro.

Tienen sentido cuando la misma fórmula aparece en muchas consultas, cuando el valor se consulta con frecuencia pero los datos base cambian poco, y cuando quieres garantizar que todos los scripts usen exactamente la misma lógica de cálculo. Agregarlas como PERSISTED tiene sentido adicional cuando el rendimiento importa y puedes beneficiarte de crear índices sobre ellas.

No tienen tanto sentido cuando la lógica de cálculo cambia frecuentemente — porque cada cambio requiere un `ALTER TABLE`, que en producción tiene su costo — ni cuando el cálculo solo se necesita en un reporte específico, donde calcularlo directamente en la query es más flexible y no afecta la estructura de la tabla.

Desafío de cierre

Agrega a la tabla `Productos` una columna calculada persistida llamada `ValorInventario` cuya fórmula sea `Precio * Stock`. Después, crea un índice sobre esa columna y escribe una consulta que encuentre los diez productos con mayor valor de inventario. Comparte tu consulta en los comentarios de la clase.